

Browser demo walkthrough — site AI inference substrate

Demo URL: `https://edge-ai-demo.vercel.app/site-inference` · works on any device, nothing to install · deterministic (add `?seed=7` to replay a different run)

This page is an **in-browser simulation** of the site-inference-substrate kit's pipeline, and it says so on screen. It is the zero-setup way to show the platform's operating behavior. The same pipeline exists as the runnable kit in this folder — real containers on a single-node cluster, proven automatically on every change (scoring, outage policies, WORM evidence, uplink sever and reconcile all asserted; see `README.md`). When the customer wants it on hardware, the kit rebuilds on a laptop in about 15 minutes (`handoff/00-partner-handoff-runbook.md`).

What is being demonstrated

Five properties of the site substrate, all visible on the page:

1. **The OT crossing is a read-only replica.** The historian feed panel is the analytics side reading exported tags (temperature, pressure, cycle counts). Nothing on this platform can write toward plant systems — there is no return path to configure.
2. **Models run at the site, behind a standard serving contract.** Three models score the looping feed. In the runnable kit they sit behind the KServe v2 inference protocol — the same contract KServe- and Triton-class runtimes speak — so the models are swappable workloads, not the platform.
3. **Per-model outage policy is configuration, visible up front.** Each model carries its policy in the table: continue, flag, or suppress. What happens during an outage is a decision engineering made in advance, per model — not improvisation during the incident.
4. **Site autonomy under uplink loss, and an honest reconcile.** Severing the uplink stops nothing: scoring continues on the local feed. Outputs queue (continue), queue marked stale-context (flag), or are held on site (suppress). On restore, queues drain and the central side reconciles — and held outputs stay held, because suppress means pending review, never auto-delivery.
5. **Evidence is signed, chained, and tamper-evident.** Bundles seal on a schedule into a write-once store, each linked to the last by hash. The Verify button proves the chain; the Tamper button alters a sealed bundle so the customer watches verification catch it.

The walkthrough

Each step: what to do, what the customer sees, what to say. The "say" lines are plain statements of what is on screen — read them or use your own words.

0 • Frame it (before touching anything) Say: "This is a simulation of how the platform behaves — the real thing is a set of containers on a single-node cluster, and I can show you that on a laptop. Watch three things: what each model is configured to do when the network drops, what happens to the outputs, and how the evidence trail holds up."

1 • Let it run (~30 seconds, no clicks) They see: three models scoring the looping feed, scored counters climbing in step, outputs delivering, evidence bundles sealing and verifying centrally. Say: "Site telemetry is being read from a replica — read-only, the analytics side can never write back — and scored right at the site by three models. Each row shows the model's outage policy: that column is configuration, agreed with engineering before anything ever goes wrong."

2 • Sever uplink They see: the link flips to SEVERED; scored counters keep climbing; outputs-queued grows; dispositions change per model — queued, queued — stale context, held for review. Say: "The connection to the operations center just dropped. The site did not stop — scoring continues on local data. Now look at the three models doing three different things: one keeps queueing normally, one queues but marks its outputs as scored-under-stale-context, and one holds its outputs on site entirely. That is policy acting, per model, exactly as configured."

3 • Let it stay severed (~30 seconds) They see: queues and held counts growing; evidence bundles still sealing on site; central counters frozen. Say: "Everything the site produces during the outage is accounted for — queued, flagged, or held. And notice the evidence stream never stopped: bundles keep sealing locally, on the chain, whether or not anyone can hear the site."

4 • Restore uplink They see: the queue drains to zero, central outputs catch up with the flagged count visible, shipped/verified evidence catches up, chain OK — and held-for-review does not move. Say: "Link is back. The site delivers everything it queued — the flagged

outputs arrive marked, so nobody mistakes them for normal-context scores. And the held outputs are still held: suppress means a person reviews them before they go anywhere. The outage ends; the policy doesn't bend."

5 • Verify chain They see: chain verification runs — every hash, link, and sequence intact. Say: "Every bundle is signed and chained to the one before it. This verification is the audit posture: not a promise, a check anyone can run."

6 • Tamper a bundle, then Verify chain again They see: a sealed bundle is altered; verification fails at exactly that bundle, naming it. Say: "Now I have altered a sealed record — and verification catches it, at the exact bundle, immediately. You cannot silently edit history on this platform. In the runnable kit the file itself also refuses rewrites; in production this lands in an object-locked bucket under your own keys."

Close on the policy table. Point at the three policies. Say: "That column is the conversation to have with your engineering and compliance owners: for each model, what should happen when a site goes dark? This platform makes that a written, versioned decision — and you just watched it hold under failure."

Questions to expect

- **"Is this the real product?"** — No, and it says so on the page. It is a simulation of the pipeline's behavior; the on-page evidence signature is a labeled stand-in. The real pipeline is the runnable kit in this folder — the same sequence runs as containers on a fresh cluster and is re-proven automatically on every change, including the write-once refusal and the chain verification. On real deployments the platform is SUSE Linux Micro + K3s or RKE2 per site class, built as one sealed Edge Image Builder image, managed by Rancher Prime + Fleet — versions in `handoff/01-component-map.md`.
- **"Whose models are these?"** — Yours, or an ISV's. The platform hosts containerized models behind the KServe v2 contract; the demo's models are illustrative stand-ins behind that real protocol surface. Swapping a model never changes the platform, the policies, or the evidence trail.
- **"Does this touch our control systems?"** — No. The analytics side reads a replica of exported tags and has no write path toward OT. Outputs are advisory — nothing here is presented to operators as a control signal.
- **"Why not just run this in the cloud?"** — The outage you just watched is the answer: scoring continued, policies held, evidence kept sealing — all site-side. Sites with poor or deliberately restricted connectivity keep working; the cloud (or the central ML platform) stays the retrain and fleet-view tier.
- **"What is the evidence stream actually for?"** — It is the audit trail: what was running (model versions, policies, applied revision), what it produced, in a signed sequence that proves nothing was altered or removed. Assembling it into a specific audit narrative is engagement work with your compliance owner.

After the demo

The next step is discovery, not a proposal: exported tag inventory and rates, the site-class map, longest outage each site must buffer through, model classes and their owners, and the evidence requirements — then agree the per-model outage policies with the engineering owner. To show it on hardware at the follow-up, run the rebuild steps in `handoff/00-partner-handoff-runbook.md` — about 15 minutes on any laptop with docker and k3d.

Source of truth: this Markdown file. The PDF beside it is generated (`tools/md2pdf.py` in this repository) — regenerate rather than editing the PDF.